

Data Structures and Algorithms – Fall 2023 Final Exam

Allowed Time: 150 minutes

Total Marks: 800

Question 1:

Code is NOT REQUIRED for this question

What is the output of the following C++ code:

```
void secretOperation(int n)
{
    if (n > 3)
        secretOperation(n - 1);
    for (int i = 3; i <= n; i++)
        cout << (n+3) % i;
    cout << endl;
}

int main()
{
    secretOperation(7);
    return 0;
}
```

Question 2:

C++ code is REQUIRED for this question

Using `unordered_map`, read multiple strings from the console as input from the user. The input ends when the user enters "stop". At the end, display all the input strings along with their counts.

Question 3:

C++ code is REQUIRED for this question

Suppose you have a **doubly circular linked list** of integers with a **tail** pointer. You are required to implement a function that should reverse every consecutive group of k nodes in the list:

```
void reverseInGroups (int k)
```

For example (assuming the values below are placed in eleven different nodes):

- Input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11] and $k = 3$
- Output: [3, 2, 1, 6, 5, 4, 9, 8, 7, 11, 10]

Note: If the number of nodes in the last group is less than k , they should also be reversed (see 10, 11).

Question 4:

C++ code is REQUIRED for this question

Implement a function to find the sum and count of even and odd values in a Binary Search Tree (BST).

Question 5:

C++ code is REQUIRED for this question

Implement a function to validate whether a given binary tree is a Binary Search Tree (BST). The function should return `true` if the given binary tree is a valid BST, and `false` otherwise. Provide the function with a prototype:

```
bool isBST(Node* root)
```

Question 6:

Code is NOT REQUIRED for this question

Consider the values given below:

23	3	8	13	18	83	153	123	173	73	63
----	---	---	----	----	----	-----	-----	-----	----	----

- Using these values, create (draw on paper) a self-balancing tree that you have studied in your DSA course. All values must be inserted in the given order.
- First add 23, then 3, 8, 13, 18, 83, 153, 123, 173, 73, and finally 63.

Questions 7 and 8:

C++ codes are is REQUIRED for these questions

- **Question 7:** Implement the **Heap Sort** algorithm to sort an array in ascending order.
- **Question 8:** For the array sorted in question 7, implement the **Binary Search** algorithm to find a target value in the sorted array. The function should return the index of the target if it exists in the array or -1 if the target is not found.